Week 2 Homework

Brandon Stonesifer

Contents

Time Series Regression Exercises **1**
A 1
B 4
Time Series Regression Responses 4

Decomposition Exercises **4**
A 5
B 8
C 10
Decomposition Responses 11

Smoothing Exercises **11**
A 11
B 12
C 13
Smothing Responses 14

Time Series Regression Exercises

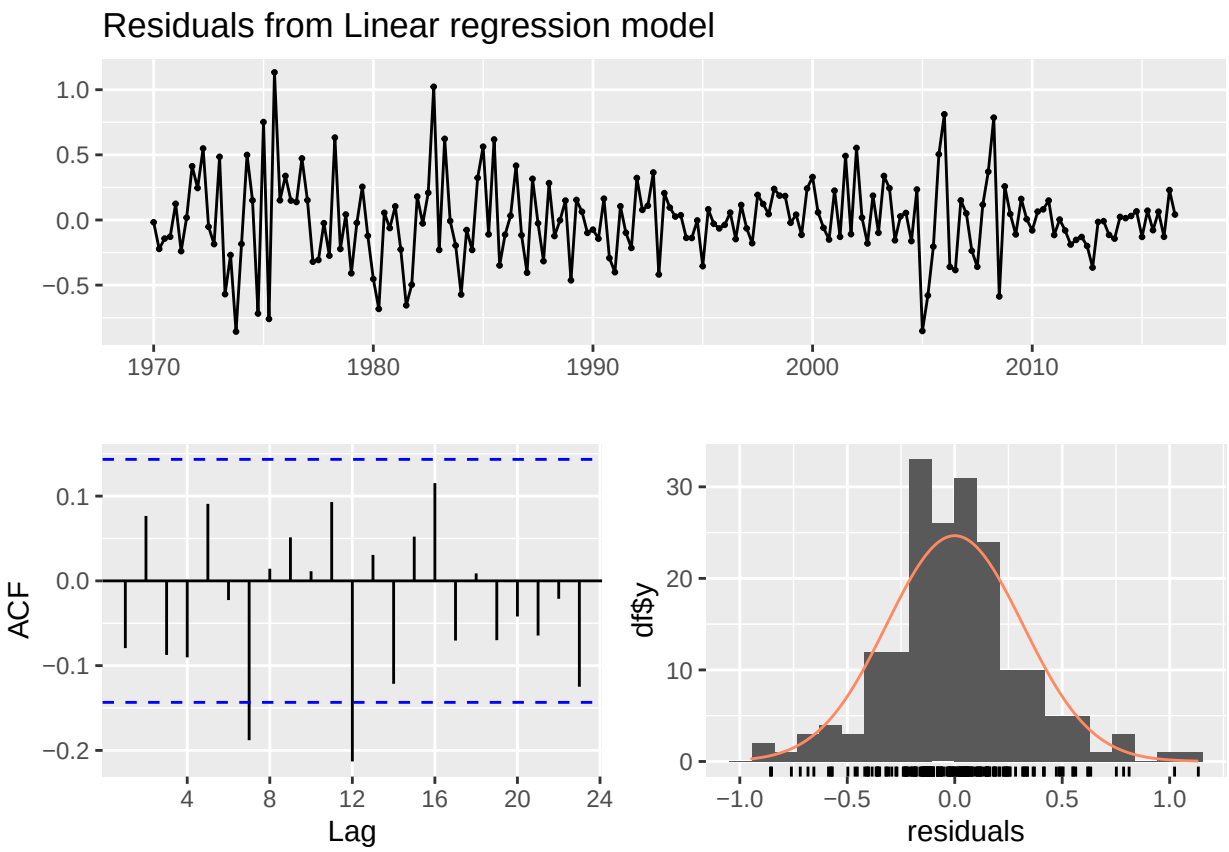
(a) Use "uschange" dataset with response, Consumption, and predictors, Income, Savings, Unemployment, trend and season, to fit a linear regression. Check the model fit with residuals, and compare how the model has changed (estimates and significance of the parameters, AIC, BIC) from what you obtained in 1 in this notebook.

(b) Use "euretail" dataset and fit a linear regression with predictors, piecewise linear trend and season. Make forecasts for the next 10 observations. Draw a plot with original response overlaid with the predicted values.

```
data(uschange)
data(euretail)
```

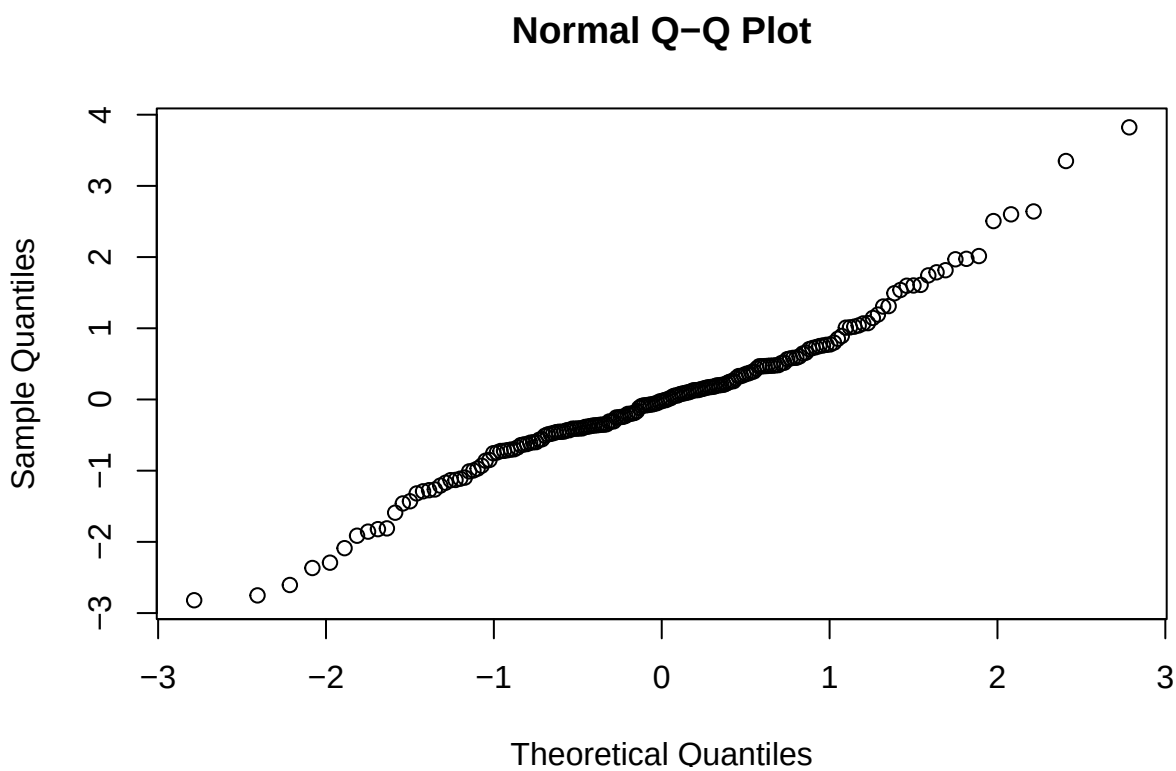
A

```
TSRE_A <- tslm(Consumption ~ Income + Savings + Unemployment + trend + season, data = uschange)
checkresiduals(TSRE_A)
```




```
##
##  Breusch-Godfrey test for serial correlation of order up to 11
##
## data:  Residuals from Linear regression model
## LM test = 19.943, df = 11, p-value = 0.04612
```

```
qqnorm(rstudent(TSRE_A))
```




```r
summary(TSRE_A)
```

```
##
## Call:
## tslm(formula = Consumption ~ Income + Savings + Unemployment +
##     trend + season, data = uschange)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.85708 -0.15452 -0.00742  0.15135  1.13327
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.4795567  0.0697559   6.875 9.97e-11 ***
## Income        0.7195019  0.0415522  17.316  < 2e-16 ***
## Savings      -0.0455217  0.0027604 -16.491  < 2e-16 ***
## Unemployment -0.3614435  0.0723238  -4.998 1.37e-06 ***
## trend        -0.0011763  0.0004484  -2.623  0.00946 **
## season2      -0.1230150  0.0670233  -1.835  0.06810 .
## season3      -0.0614825  0.0674105  -0.912  0.36296
## season4      -0.1387327  0.0673973  -2.058  0.04100 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.323 on 179 degrees of freedom
## Multiple R-squared:  0.7662, Adjusted R-squared:  0.757
## F-statistic: 83.78 on 7 and 179 DF,  p-value: < 2.2e-16
```

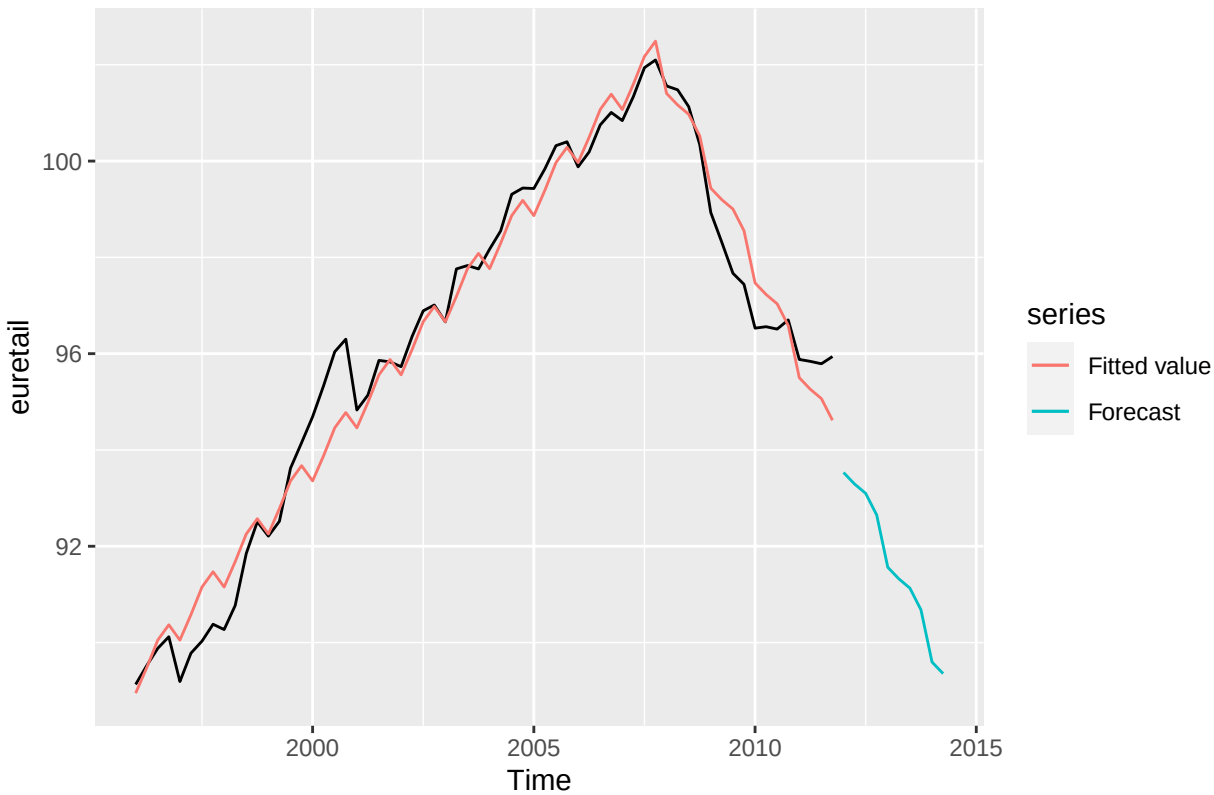
```r
c(AIC = AIC(TSRE_A), BIC = BIC(TSRE_A))
```

```
##     AIC     BIC
```

```
## 117.899 146.979
```

B

```
TSRE_B <- tslm(euretail ~ trend + season)
fit <- tslm(euretail ~ trend + I(pmax(trend - 48, 0))+ season)
TSRE_B2 <- forecast(fit, h = 10)
autoplot(euretail)+
  autolayer(fitted(fit), series = "Fitted value")+
  autolayer(TSRE_B2$mean, series = "Forecast")
```



Time Series Regression Responses

- (a) The model fits reasonably well, with intuitive and strongly significant effects for Income, Savings, and Unemployment. However, residual diagnostics show remaining autocorrelation, meaning the model's standard errors and significance levels may be overly optimistic. Overall fit metrics (AIC/BIC) are acceptable, but the autocorrelation suggests the model could be improved by addressing time-series dependence.
- (b) Forecasted a continue decrease consistent with the current trend.

Decomposition Exercises

1. Use built-in AirPassengers dataset in R.

(a) Apply additive and multiplicative classical decomposition methods. For each method, draw a plot where data, seasonal adjusted estimate, trend and trend+seasonal (trend*seasonal for multiplicative method) are overlaid. Which method produces better results, and why?

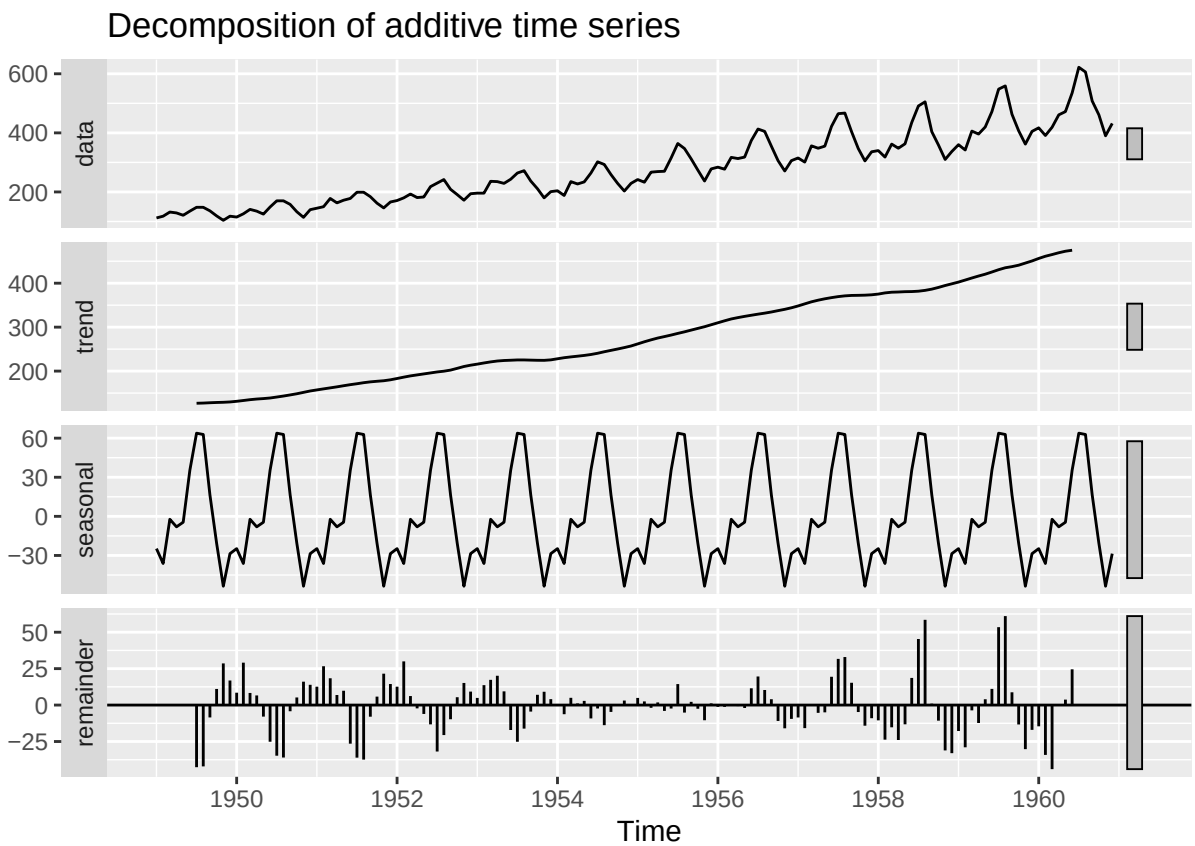
(b) Apply other decomposition methods, X11 method and STL decomposition methods. For each method, draw a plot where data, seasonal adjusted estimate, trend are overlaid. Compare the results of the two methods and also the classical methods from (a).

(c) Compute seasonal strength F_s from each method you applied in (a-b).

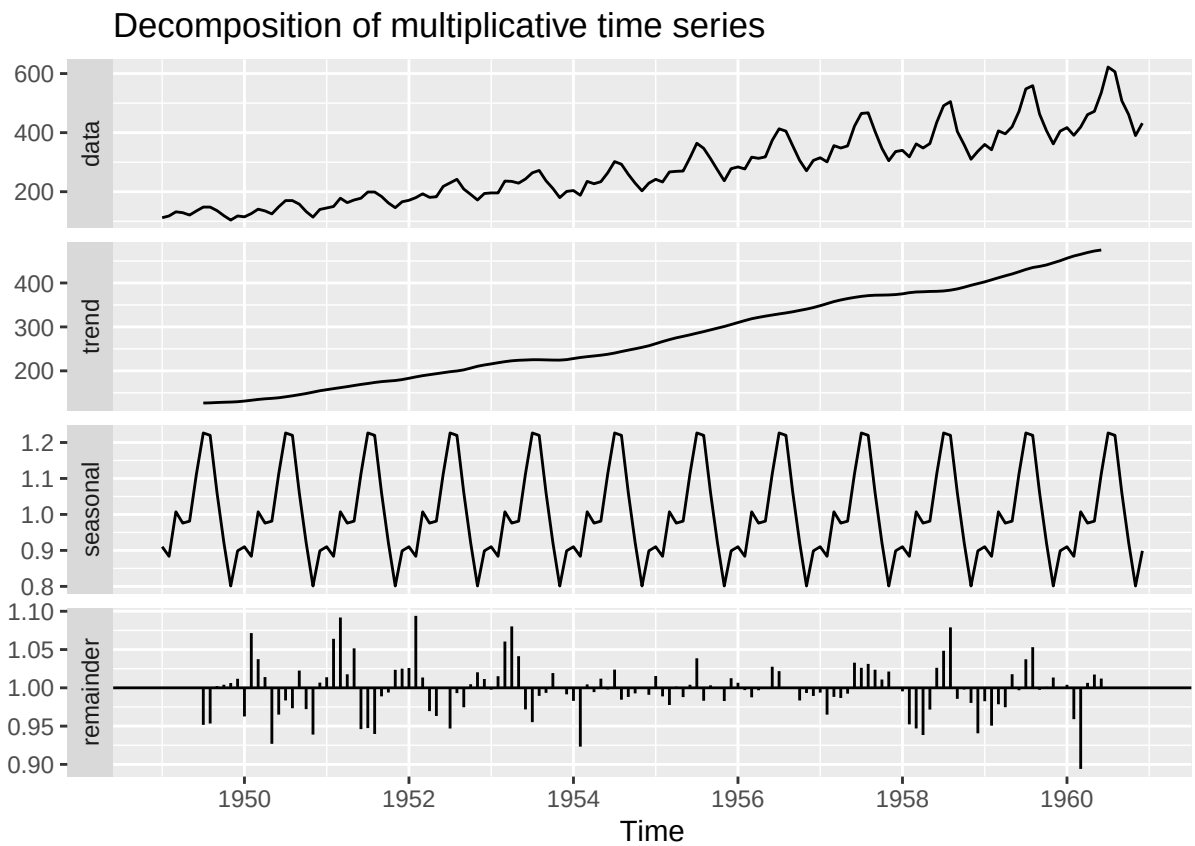
```
data(AirPassengers)
```

A

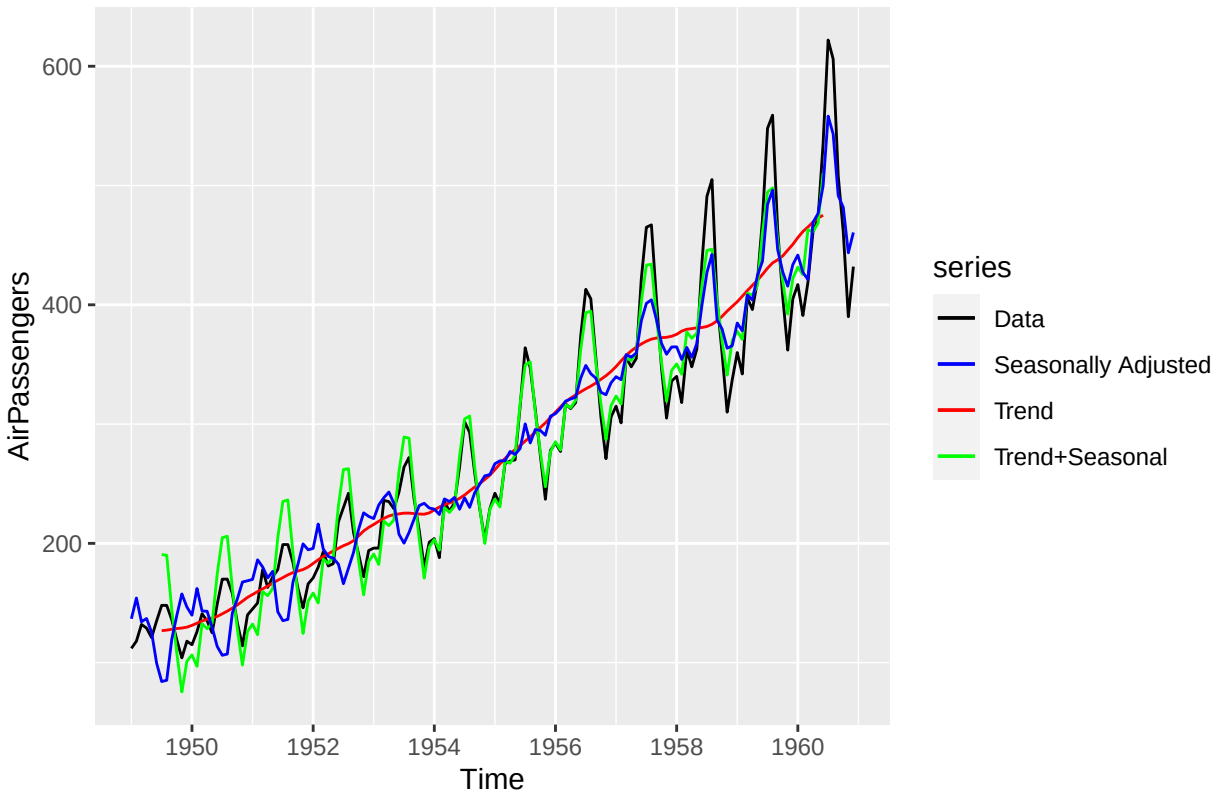
```
DE_AA <- decompose(AirPassengers, type = "additive")
DE_AM <- decompose(AirPassengers, type = "mult")

autoplot(DE_AA)
```



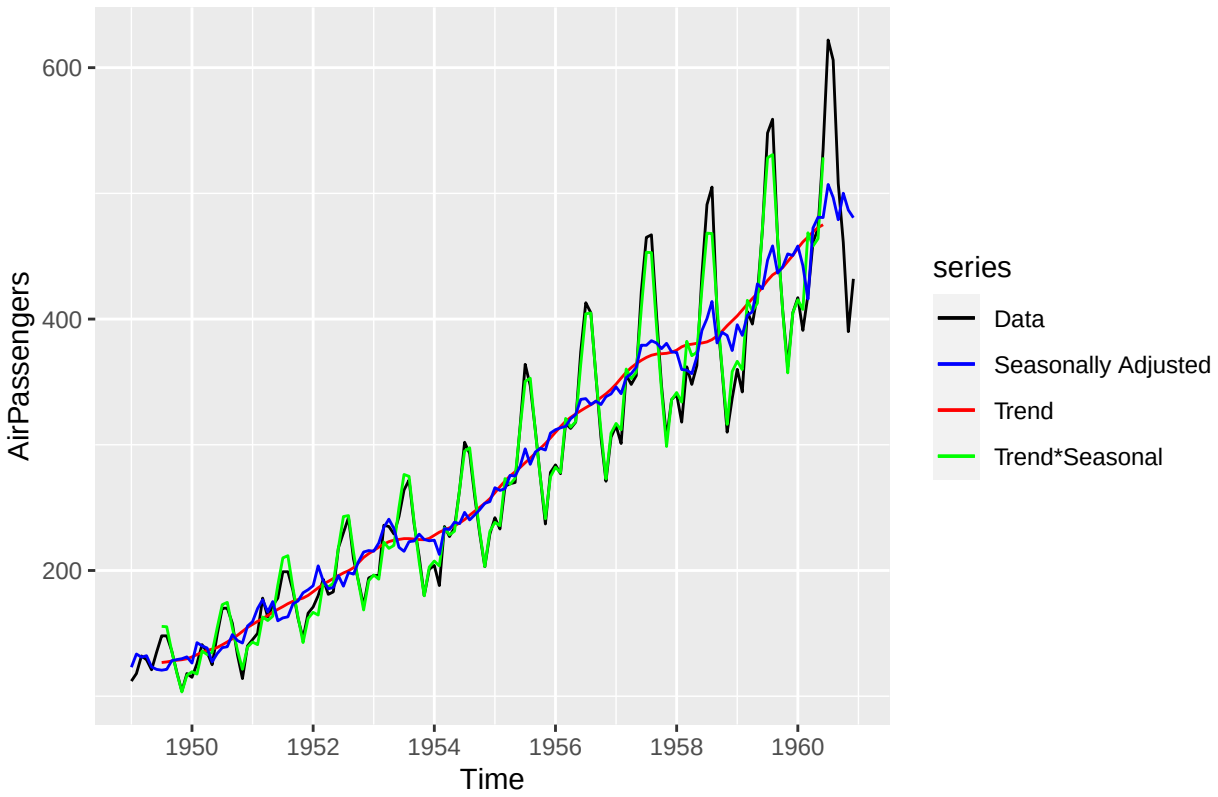

```
autoplot(DE_AM)
```




```
autoplot(AirPassengers, series="Data") +
  autolayer(trendcycle(DE_AA), series="Trend") +
  autolayer(trendcycle(DE_AA)+seasonal(DE_AA), series="Trend+Seasonal") +
  autolayer(seasadj(DE_AA), series="Seasonally Adjusted") +
  scale_colour_manual(values=c("black","blue","red","green"),
      breaks=c("Data","Seasonally Adjusted","Trend","Trend+Seasonal"))
```

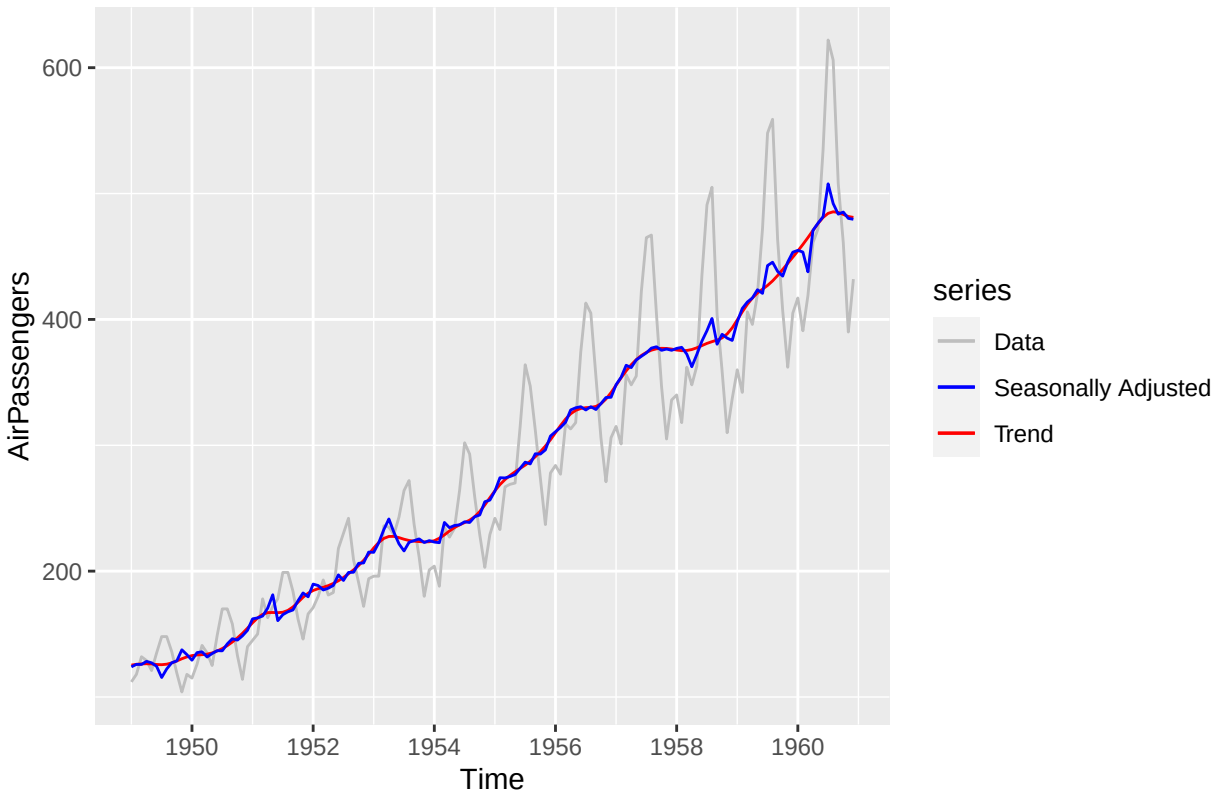



```
autoplot(AirPassengers, series="Data") +
  autolayer(trendcycle(DE_AM), series="Trend") +
  autolayer(trendcycle(DE_AM)*seasonal(DE_AM), series="Trend*Seasonal") +
  autolayer(seasadj(DE_AM), series="Seasonally Adjusted") +
  scale_colour_manual(values=c("black","blue","red","green"),
      breaks=c("Data","Seasonally Adjusted","Trend","Trend*Seasonal"))
```

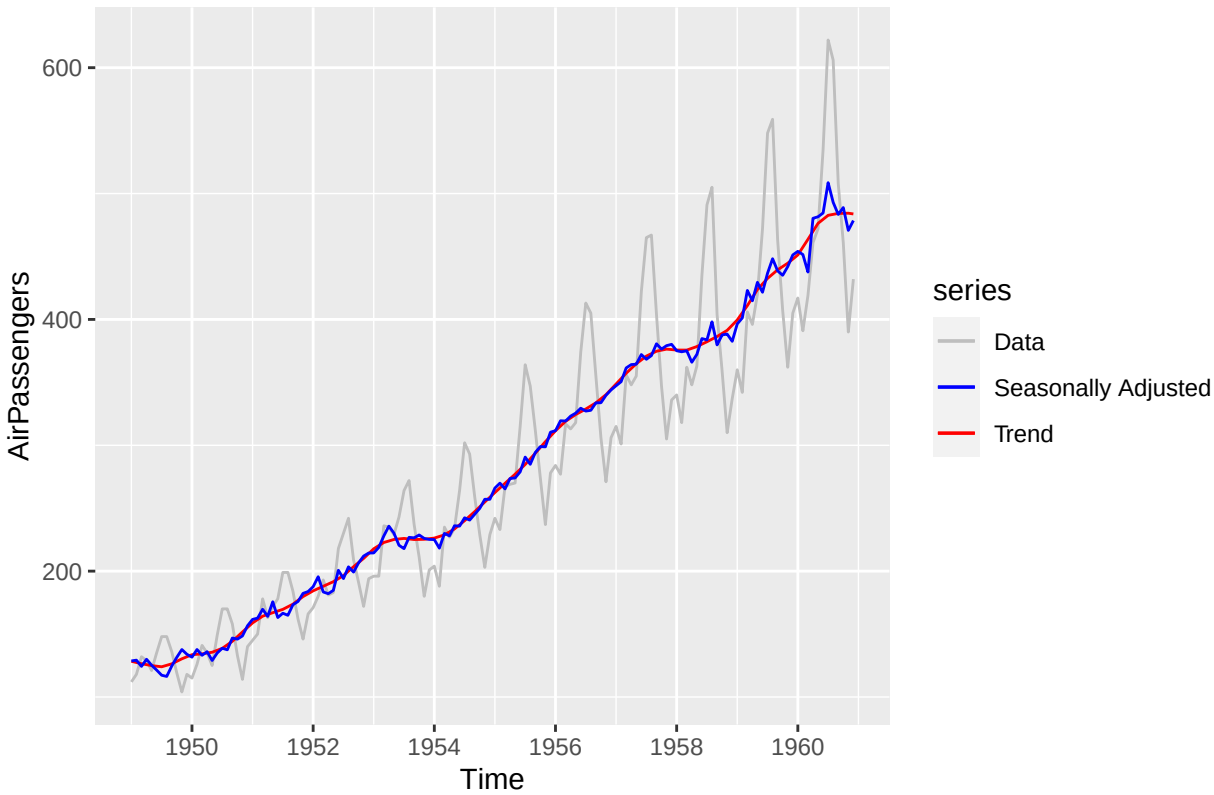


B

```
DE_x11 <- seasonal::seas(AirPassengers, x11 = "", transform.function = "none")
DE_STL <- stl(AirPassengers, s.window = 7, t.window = 13)
autoplot(AirPassengers, series="Data") +
  autolayer(trendcycle(DE_x11), series="Trend") +
  autolayer(seasadj(DE_x11), series="Seasonally Adjusted") +
  scale_colour_manual(values=c("grey","blue","red"),
      breaks=c("Data","Seasonally Adjusted","Trend"))
```




```
autoplot(AirPassengers, series="Data") +
  autolayer(trendcycle(DE_STL), series="Trend") +
  autolayer(seasadj(DE_STL), series="Seasonally Adjusted") +
  scale_colour_manual(values=c("grey","blue","red"),
      breaks=c("Data","Seasonally Adjusted","Trend"))
```



C

```
DE_x11_strength <- (1 - var(remainder(DE_x11))/var(seasonal(DE_x11) +
                                                remainder(DE_x11))) %>% max(0, .)

DE_STL_strength <- (1 - var(remainder(DE_STL))/var(seasonal(DE_STL) +
                                                remainder(DE_STL))) %>% max(0, .)

DE_AA_strength  <- (1 - var(remainder(DE_AA), na.rm=TRUE)/var(seasonal(DE_AA) +
                                                remainder(DE_AA), na.rm=TRUE)) %>% max(0, .)

DE_AM_strength  <- (1 - var(remainder(DE_AM), na.rm=TRUE)/var(seasonal(DE_AM) +
                                                remainder(DE_AM), na.rm=TRUE)) %>% max(0, .)


Strengths <- rbind(
  X11 = DE_x11_strength,
  STL = DE_STL_strength,
  Additive  = DE_AA_strength,
  Miltiplitve  = DE_AM_strength
)

colnames(Strengths) <- "Seasonality Strength"
Strengths <- round(Strengths, 3) * 100
Strengths
```

```
##             Seasonality Strength
## X11                         98.5
```

```
## STL                          98.3
## Additive                     77.9
## Miltiplitve                  93.6
```

Decomposition Responses

- (a) Multiplicative produces better results than additive because multiplicative allows the model to capture seasonality that scales rather than stays constant.
- (b) Both X11 and STL perform better because X-11 and STL can flexibly estimate seasonality and trends that change over time, they adapt better to real-world data than a fixed multiplicative structure, which assumes seasonality scales strictly with the level and can't adjust when patterns shift.

Smoothing Exercises

1. Use built-in AirPassengers dataset in R.

(a) Draw a plot where training data, testing data, forecast from the four methods are overlaid. Visually examine the plot, and describe the results. Which method produces the best forecasting results? Why do you think that method was most effective?

(b) Compare the four methods by accuracy measures (MAE, RMSE, MAPE, MASE) using forecasting error. Which method produces the best forecasting results according to which accuracy measure?

(c) Apply `ets()` function to the training dataset (do not specify the model so that R can choose it), obtain the forecast for the next 12 observations, and accuracy measures of the method. Draw a plot where training data, testing data, and forecasts from the five methods in (a,c) are overlaid. Also, compare the results from accuracy measures. Which method produces the best results? Why do you think that method was the most effective?

```r
data(AirPassengers)
```

A

```r
AP <- AirPassengers
frequency(AP)
```

```
## [1] 12
```

```r
start(AP); end(AP)
```

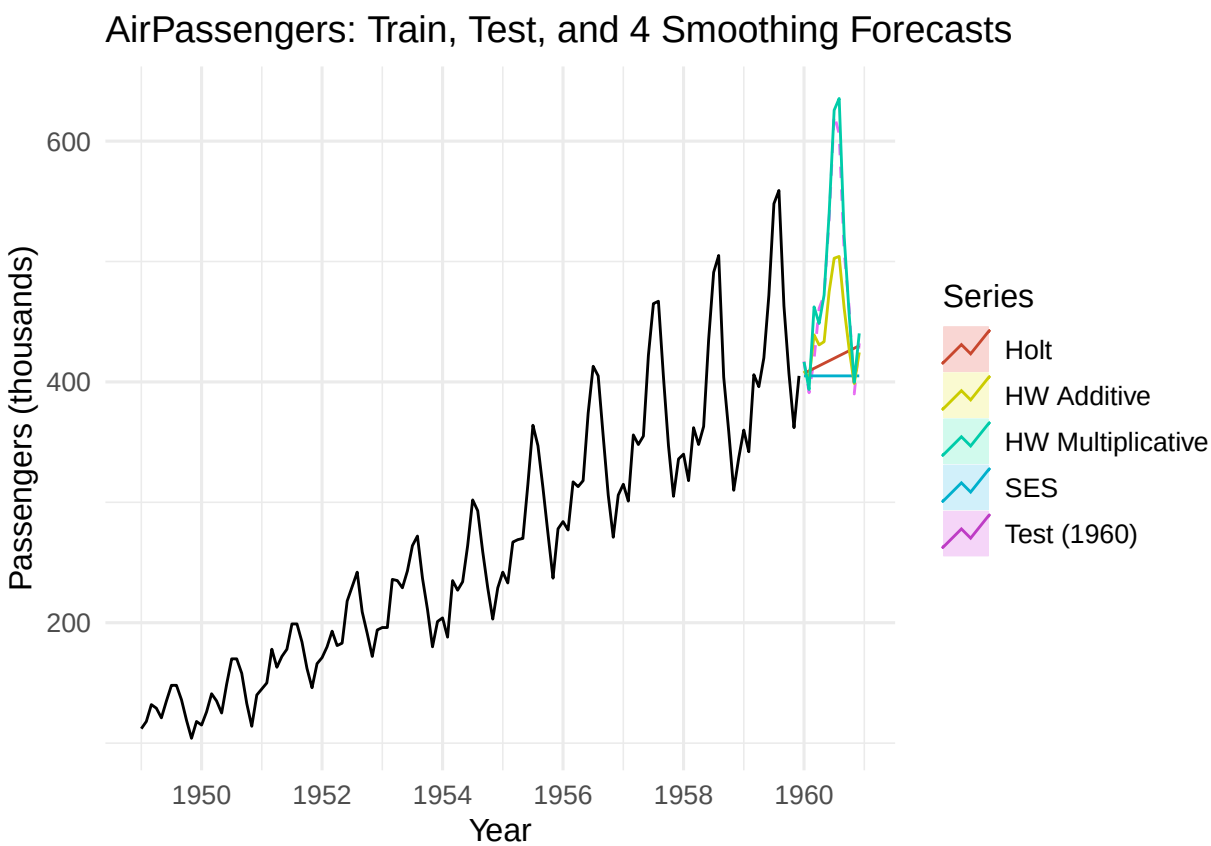
```
## [1] 1949    1
```

```
## [1] 1960   12
```

```r
train <- window(AP, end = c(1959, 12))
test  <- window(AP, start = c(1960, 1))
h <- length(test)
```

```r
# SES
fit_ses  <- ses(train, h = h)

# Holt's linear trend (no seasonality)
fit_holt <- holt(train, h = h)

#  Holt-Winters Additive
fit_hwa  <- hw(train, seasonal = "additive", h = h)
```

```r
#  Holt-Winters Multiplicative
fit_hwm  <- hw(train, seasonal = "multiplicative", h = h)

# Plot overlay: train, test, and the 4 forecasts
autoplot(train) +
  autolayer(test, series = "Test (1960)", linetype = "dashed") +
  autolayer(fit_ses,  PI = FALSE, series = "SES") +
  autolayer(fit_holt, PI = FALSE, series = "Holt") +
  autolayer(fit_hwa,  PI = FALSE, series = "HW Additive") +
  autolayer(fit_hwm,  PI = FALSE, series = "HW Multiplicative") +
  labs(title = "AirPassengers: Train, Test, and 4 Smoothing Forecasts",
       x = "Year", y = "Passengers (thousands)") +
  guides(colour = guide_legend(title = "Series")) +
  theme_minimal(base_size = 12)
```



B

```r
models_b <- list(
  SES                 = fit_ses,
  Holt                = fit_holt,
  `HW Additive`       = fit_hwa,
  `HW Multiplicative` = fit_hwm
)

acc_b <- do.call(
  rbind,
```

```r
  lapply(models_b, function(m) as.data.frame(accuracy(m)))
)

acc_b[, c("RMSE", "MAE", "MAPE", "MASE")]
```

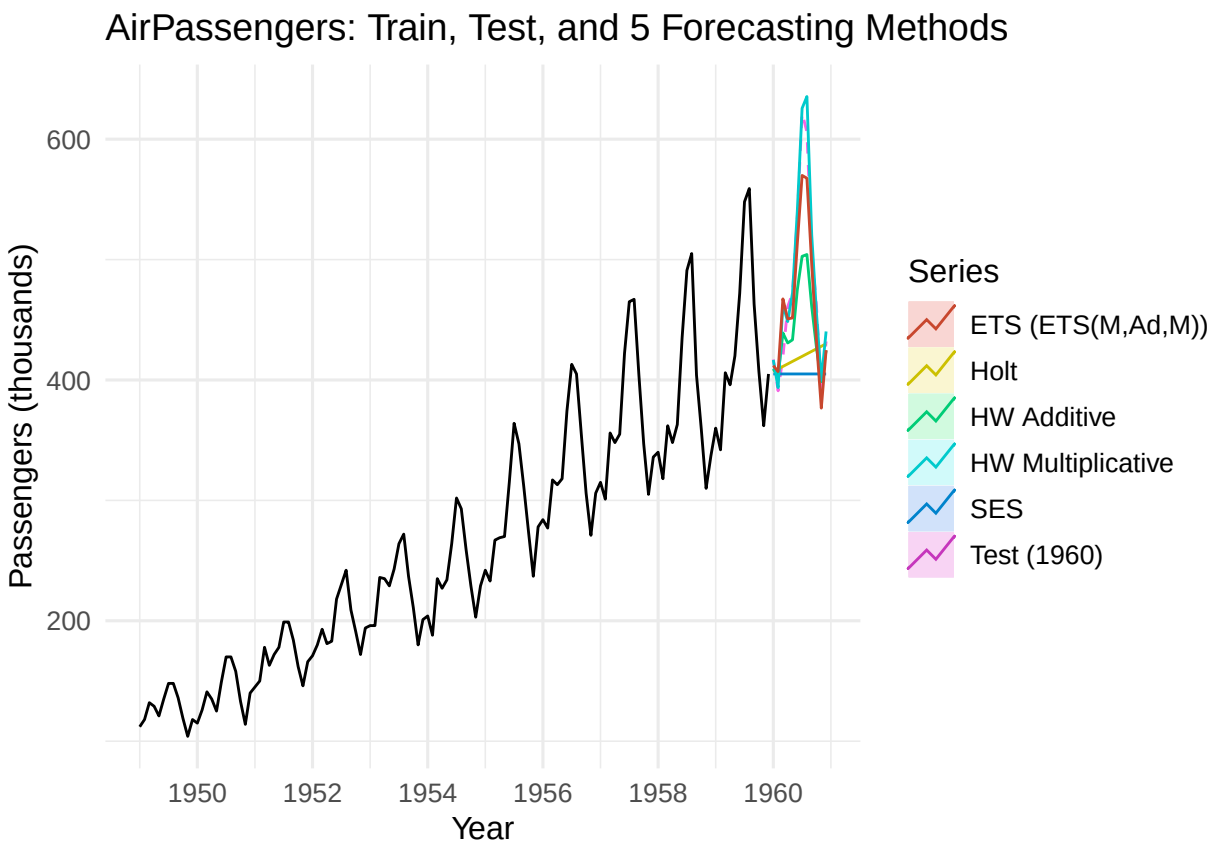
```
##                        RMSE       MAE     MAPE      MASE
## SES               31.214124 23.902285 8.911792 0.7849683
## Holt              31.151717 23.829502 8.965372 0.7825780
## HW Additive       15.410830 11.572039 5.002286 0.3800341
## HW Multiplicative  9.949946  7.533284 2.997750 0.2473985
```

C

```r
# Auto ETS on training data
fit_ets <- ets(train)
fit_ets
```

```
## ETS(M,Ad,M)
##
## Call:
## ets(y = train)
##
##   Smoothing parameters:
##     alpha = 0.758
##     beta  = 0.0213
##     gamma = 1e-04
##     phi   = 0.98
##
##   Initial states:
##     l = 120.7483
##     b = 1.7632
##     s = 0.897 0.798 0.919 1.0587 1.2156 1.2251
##            1.1075 0.9782 0.9804 1.0207 0.8926 0.9073
##
##   sigma:  0.0378
##
##      AIC     AICc      BIC
## 1244.458 1250.511 1296.348
```

```r
fc_ets  <- forecast(fit_ets, h = h)

# Plot
autoplot(train) +
  autolayer(test,   series = "Test (1960)", linetype = "dashed") +
  autolayer(fit_ses,  PI = FALSE, series = "SES") +
  autolayer(fit_holt, PI = FALSE, series = "Holt") +
  autolayer(fit_hwa,  PI = FALSE, series = "HW Additive") +
  autolayer(fit_hwm,  PI = FALSE, series = "HW Multiplicative") +
  autolayer(fc_ets,   PI = FALSE, series = paste0("ETS (", fit_ets$method, ")")) +
  labs(title = "AirPassengers: Train, Test, and 5 Forecasting Methods",
       x = "Year", y = "Passengers (thousands)") +
  guides(colour = guide_legend(title = "Series")) +
  theme_minimal(base_size = 12)
```




```r
# Accuracy table

models <- list(
  SES = fit_ses,
  Holt = fit_holt,
  `HW Additive` = fit_hwa,
  `HW Multiplicative` = fit_hwm,
  `ETS (auto)` = fc_ets
)

acc_all <- do.call(rbind, lapply(models, function(m) as.data.frame(accuracy(m))))

acc_all[, c("RMSE", "MAE", "MAPE", "MASE")]
```

```
##                        RMSE       MAE     MAPE      MASE
## SES                31.214124 23.902285 8.911792 0.7849683
## Holt               31.151717 23.829502 8.965372 0.7825780
## HW Additive        15.410830 11.572039 5.002286 0.3800341
## HW Multiplicative   9.949946  7.533284 2.997750 0.2473985
## ETS (auto)          9.353686  6.980909 2.761156 0.2292581
```

Smothing Responses

- (a) From the overlay, Holt-Winters Multiplicative visually tracks the seasonal peaks/troughs and rising level best among the four; SES/Holt miss seasonality, HW Additive underestimates peak amplitudes.
- (b) Using MAE, RMSE, MAPE, MASE, the HW Multiplicative method produces the best (lowest) scores among the four.

- (c) Adding ets() (auto) usually edges out HW Multiplicative across most accuracy metrics (especially MAPE/MASE), because it can select multiplicative errors, multiplicative seasonality, and damped trend, optimizing parameters and model form via AIC.